March 2nd 2026

Reinforcement learning policies for Formula 1 gym

STAT 4830

Yuv Malik | Matthew Lobo | Pablo Echevarria Cuesta

Version 1.0

Problem Statement

"ELEVATOR PITCH"

A RL project where we train an agent to drive a race car autonomously in simulation. The agent sees the world through 96x96 RGB images from the environment and learns to output continuous control actions (gas, steering, and brakes).

WHAT DO WE OPTIMIZE FOR EXACTLY?

We want to maximize the expected cumulative reward. The reward function is designed to: make forward progress around the track, stay on the road, and avoid crashes.

END GOAL: 3D RACING

Ultimately, we want the agent to race against multiple competitors and can generalize to any track such that it can drive competitively in any track. In 3D, the physics gets more realistic and hard (e.g. collisions, tire traction, weight transfer, etc.)

Technical Approach

PYTORCH

Architecture: A shared 3-layer CNN encoder feeding into separate MLP heads (256 → 128 units) for the Actor and Critic.

- Policy Network Learns distribution
- Value Network learns the expected return under a policy

$$L_{value} = (V_\phi(s_t) - R_t)^2$$

Maths:

$$\nabla J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) A(s,a)]$$

Policy Updates:

- Using log probability for calculus convenience
- Grad weighted with Advantage (only updates with surprise)
- Calculate advantage comparing current policy vs value head
- Expectation (as we are training in batches)

VALIDATION

Performance Metrics: Real-time tracking of Mean Reward, % Lap Progress, and Off-track Rate. Tensorboard (clipping, ratios etc)

Regression Testing: Benchmarking custom implementation against Stable-Baselines3 (SB3) performance (~282$ reward baseline).

Failure Analysis: Iterative debugging of "Reward Hacking" (exploiting velocity on grass) and "Donut Behavior" via visual evaluation and telemetry logs.

CONSTRAINTS

Compute: Optimized for ~500k to 1M training steps per run.

Running on Local GPU on PREDATOR HELIOS NEO 14 with an NVIDIA GEFORCE RTX.

Hyperparameters: Learning rate (2.5 x10^-4), batch size 1024, and GAE lambda 0.95.

Operational Constraints: Limited by the "FailFast" trigger and the need for high-fidelity progress tracking to avoid non-meaningful reward inflation.

Technical Approach

REWARDS

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

Full PPO Objective:

- r theta is equal to the ratio of the new policy over the old policy
- The important part is that we have clipped here to prevent over aggressive policy updates
- Value of future rewards

Reward Function: Primary signal is actual tile progress delta , augmented by speed bonuses (on-track only) and penalized for yaw rate (donuts), idling, and steering instability.

Constraints: Environment terminates (episode ends) if the car goes off-track, stays stuck ($speed < 0.8$), or makes zero progress for 200 steps.

$$r_t = \underbrace{\alpha_{\text{fwd}}\,\Delta p_t}_{\text{progress}} + \underbrace{\alpha_{\text{spd}}\, s_t\, 1_{\text{on}}(t)\, 1_{\neg\text{corner}}(t)}_{\text{straight speed}}$$

$$+ \underbrace{-\alpha_{\text{smooth}} \min(|\delta a_t^{\text{steer}}|,\, c_\delta)}_{\text{steer smoothness}} + \underbrace{-\alpha_{\text{mag}}(a_t^{\text{steer}})^2}_{\text{steer magnitude}}$$

Initial Results

First Rendition

- Utilized Stable Baselines (SB3) for racetrack environment
- Spins in circles, no real forward progress
- Reward exploited + unstable policy and value networks
- Code issues:
  - Policy and value network shared some layers (gradient/weight interference)
    - Same weights are pushed in different directions, leading to unstable and slower learning
  - Velocity-based reward (dot(v, track_dir))
    - Dot product tracks direction, but can't tell whether or not the car is on or off the track
    - So if the car is going in the same direction as the track, but it is off the track, it is falsely rewarded, which is problematic.
  - Result: High reward, 0% actual progress

link: https://drive.google.com/drive/folders/1GxcDMC381MmItWLdJlegAE-fLZLj4Tlj

Second Rendition

- Used PyTorch entirely instead of Stable Baselines (SB3)
  - Gives us more control and a better understanding of the lower-level code
- More controlled, but speed is a lot slower
- Moves forward, better turns for large corners, but goes slightly off the track
- Separated policy & value networks (no more gradient/weight interference)
- Only real track progress is rewarded
  - In order to be rewarded, car must make forward progress while staying on the track.
  - Improvement from the velocity-based reward (dot(v, track_dir)), which couldn't tell whether or not the car was on or off the track
- Off-track now ends episode
- Result: Much more stable driving

Third Rendition

- Faster, makes real progress
- Goes fast straight on the track, but unstable on sharp turns
- Spins on sharp turns
- Code additions:
  - Yaw-rate penalty (−0.1 × |angular velocity|)
    - a penalty for excessive rotation, so the car gets punished if it spins too much
  - No-progress termination
    - End the episode if the car isn't making forward progress for a while
  - Speed-based corner penalties
    - Car gets punished for fast, sharp turns

Next Steps

COMPLETE A FULL LAP

Look to complete a whole lap on track. Then we need to look to start optimizing further for racing line.

ADD ANOTHER AGENT

Initially, we will look to add an agent that has a preset line. Our agent will adapt to movement of a preset Computer User/Car.

START A RACE

Race with 20 cars, learn to navigate much more complex agent environments.